

Backend Requirements Addendum

Volunteer Organization Platform — Delta vs. Backend-Requirements-Document.pdf

Reference document: `docs/Backend-Requirements-Document.pdf` (dated 2026-08-09)

Source of truth: fresh static analysis of the current React/Vite frontend codebase (`c:\Users\hp\Desktop\volunteer-organization`), same methodology as the reference document — services, hooks, validation schemas, constants, components. File citations (`path:line`) point at the current code.

Scope: this file contains only what is new, changed, or contradictory since the reference document. Anything in the reference document that still matches the current code is intentionally omitted — see the reference document itself for everything unchanged.

Section numbers below (§) match the reference document's numbering so the two can be merged directly.

Revision note (2026-08-14): the entries marked **UPDATE (2026-08-14)** below were added across two follow-up code change sessions the same day (notification read-state handling; then the applicant email field, §17.4). Everything else in this document is unchanged from the original pass.

§3.8 Governorate/City — data now populated, naming ambiguity resolved

NEW — `nameAr` is now populated with real Arabic names for all 14 seed governorates (previously the field existed in the data model per the reference doc but held no data anywhere in the static list). Usable directly as backend seed data:

`src/services/syrianGovernorates.js:16-31`

CHANGED / RESOLVED — the reference document's §23 flagged unresolved "naming drift" between "Rural Damascus" and "Rif Dimashq" across 4 duplicated governorate lists. This is now resolved app-wide, consistently, everywhere a governorate/city value is actually submitted: the canonical governorate is displayed as "Rural Damascus" / "Rif Dimashq (Damascus Countryside)", but the value transmitted to the API for that governorate is uniformly the string `"Rif Dimashq"` — never `"Rural Damascus"` — in every form:

Location

File:line

Volunteer profile validation (enum)	<code>src/utils/auth/VolunteerProfileValidation.js:29</code>
Volunteer profile form (select value)	<code>src/components/volunteerProfile/ProfileForm.jsx:27, 40</code>
Organization profile validation (enum)	<code>src/utils/auth/OrganizationProfileValidation.js:18, 24-25</code>
Organization profile form (select value)	<code>src/components/OrgProfile/ProfileForm.jsx:20-22</code>
Opportunity creation form (select value)	<code>src/components/organization/CauseForm.jsx:12-14</code>

Backend action: the `city` / `location` enum accepted on volunteers, organizations, and opportunities must accept `"Rif Dimashq"` as the value for this governorate — not `"Rural Damascus"`. This applies identically to all three entities now (previously an open inconsistency; now a single deliberate, unified choice on the frontend).

§6 Route Map — two new routes, one legacy redirect

Path	Access	Page
<code>/my-journey</code>	Volunteer only	NEW — replaces the standalone achievements page: hours summary, achievements, and a completed-opportunities timeline, all in one page (<code>src/pages/volunteerJourney.jsx</code>). No new API calls — reuses existing participations/achievements queries.
<code>/notifications</code>	Any authenticated user	NEW — full notification history/filter page (<code>src/pages/notifications.jsx</code>), reached via "See all notifications" from the navbar bell.

Legacy redirect added: `/achievements` → `/my-journey` (`src/constants/paths.js:13,21,39`).

DOC/CODE MISMATCH — the reference document's §6 lists the applicants route as `/applicants/:id`. The actual registered route (unchanged, already true when the reference document was written) is `/mycauses/applicants/:id` (`src/constants/paths.js:17` , `src/App.jsx:95`). Worth correcting in the merged route map.

§14 Notifications — catalogue grew, one endpoint now concrete

The reference document described this system as "approved by backend but not built" with only a hypothetical shape. The frontend notification system has since been substantially built out. What's new

relative to the reference document:

NEW notification type — `applicant-new`: "New volunteer application" — fires for an organization when one of its opportunities has an unseen `pending` participation (a volunteer just applied). Links to `/mycauses/applicants/{opportunityId}`, uses the same seen-tracker as `applicant-withdrawn` (`organizationApplicantSeenTracker.js`).

`src/services/notifications.js:184-192` (generation) ·

`src/constants/notificationTypes.js:17,29,43` (icon/color/label)

This brings the full notification type catalogue to **9 types** (the reference document's §14 listed 8, missing this one): `achievement`, `hours`, `status-accepted`, `status-rejected`, `org-verified`, `org-rejected`, `applicant-new`, `applicant-withdrawn`, `opportunity-reminder`.

NEW — concrete endpoint, not just a hypothetical shape — the real-mode client now calls a specific mark-as-read endpoint on dismiss:

`POST /notifications/{id}/read` — `src/services/notifications.js:278`

The reference document's §14 only speculated "a `PATCH /notifications/{id}/read`-style mechanism will be needed." The frontend has since committed to a concrete verb and path: `POST`, not `PATCH`. The main listing endpoint (`GET /notifications`) is still unbuilt server-side — unchanged from the reference document.

NEW — "Mark all as read" (`pages/notifications.jsx`) does not call a bulk endpoint. It loops over every visible item and fires the same `POST /notifications/{id}/read` call once per item (`src/pages/notifications.jsx:76-83`). If a volunteer/organization has many unseen notifications, this means *N* parallel `POST` requests, not one. Worth a bulk `POST /notifications/read-all` if the backend team wants to avoid that, but it is not required by the current frontend contract.

UPDATE (2026-08-14) — client-side unread filtering added; a real bug in the real-mode branch is now fixed. Previously the real-mode client read a `seen` field back from the (still hypothetical) `GET /notifications` response but never actually used it to filter — every notification the backend ever returned, read or unread, would have counted toward the bell badge and appeared in the dropdown. The client now:

- sends `?unread=true` optimistically on `GET /notifications` (mirrors the same optimistic-query-param pattern already used for `?status=verified` on `fetchOrganizations`, `src/services/organizations.js:205`), and
- defensively filters out any item with `seen: true` client-side regardless of whether the backend honors that param (`data.filter((item) => !item.seen)`), so the returned list is guaranteed

unread-only either way — matching the mock-mode branch's existing behavior of only ever returning unseen items.

`src/services/notifications.js:258-282`

UPDATE (2026-08-14) — mark-as-read now also fires automatically on click-through, not only on explicit dismiss. Clicking a notification to navigate to its linked page now calls the same `onDismiss` handler that the "X" dismiss button and "Mark all as read" already used. In real mode this means `POST /notifications/{id}/read` fires on every notification click-through in addition to explicit dismissal — same endpoint and contract as already documented above, just a higher expected call frequency in practice. No new endpoint is required.

`src/components/notifications/NotificationListItem.jsx:27-33`

Backend action: optionally support an `?unread=true` query parameter on `GET /notifications` to return only unread notifications server-side (cheaper than returning full history every poll). Not required — the frontend already filters defensively client-side if the parameter is ignored or unsupported — but recommended given the client polls this endpoint every 5 seconds while a user session is active (`src/hooks/useRecentUpdates.js:23`).

§15 Dashboards & Analytics (Organization) — recentActivity filter changed

CHANGED — the reference document's §15 defines `recentActivity` in the expected `GET /organizations/me/dashboard` response as "top 5 by date desc" with no status filter. The frontend's client-side composition of this field now filters to **pending-status applicants only** before taking the top 5 — the intent being to surface what still needs the organization's action, not a general activity log that includes already-decided (accepted/rejected) applications.

`src/services/dashboard.js:291-305`

Backend action: when `GET /organizations/me/dashboard` is implemented, `recentActivity` should be the 5 most recent **pending** participations for the organization's opportunities, ordered by `participatedAt` / `date` descending — not the 5 most recent participations of any status.

§17.4 Applicants list — new required field: `volunteer.email`

UPDATE (2026-08-14) — the applicant card and the volunteer profile preview modal (both reached from the organization's "Applicants" page) now display the volunteer's email address next to their

phone number, using the same treatment already given to `volunteer.phone` and `volunteer.photo` in this same response: read directly off the nested `volunteer` object, rendered only when present, no separate lookup call.

Why: organizations reviewing applicants currently only see a phone number as a contact method. Email is a second, equally direct contact channel, and every volunteer account already has one (it's the login identifier, §3.1 of the reference document) — so no new data collection is required, only exposing a field that already exists on the user record.

Exact field name: `email`, nested inside the `volunteer` object of each item in the `GET /opportunities/{id}/participants` response — i.e. `volunteer.email`, sibling to the already-documented `volunteer.phone` and `volunteer.photo` fields (§16 of the reference document), not a top-level field on the participation record itself.

Backend action: add `email` to the nested `volunteer` object returned by `GET /opportunities/{id}/participants` (sourced from `users.email` for that participation's volunteer). No renaming, no normalization layer, and no new frontend logic is involved on this side — `src/services/participations.js` was deliberately left untouched; the field is expected to arrive already shaped inside the nested `volunteer` object, exactly like `volunteer.phone` does today. If the field is absent or empty, the frontend simply omits the row (same defensive pattern already used for `volunteer.city` and `volunteer.phone`) — so this is safe to ship incrementally and won't break anything if rolled out before the rest of the participants endpoint is finished.

`src/components/organization/ApplicantCard.jsx:93-98` ·

`src/components/organization/VolunteerProfilePreviewModal.jsx:115-123` · unchanged:

`src/services/participations.js:129-131`

§19 Full Endpoint Catalog — addition

Method & Path	Request	Response	Status
<code>POST /notifications/{id}/read</code>	—	success/error	NEW (verb/path now committed in frontend code, see §14)

GET /notifications	optional ? unread=true	array of notification objects (id, type, title, description, href, seen)	UPDATE (2026-08-14) — query param is a frontend addition, optional server-side (see §14 update). Endpoint itself still unbuilt.
GET /opportunities/{id}/participants	—	applicant list; nested volunteer object must now include email	UPDATE (2026-08-14) — volunteer.email is a new required field on this response, see §17.4 update.

§22 Backend Team Checklist — additions

- Ensure the governorate/city enum accepts "Rif Dimashq" (not "Rural Damascus") as the value for that governorate, consistent across volunteers, organizations, and opportunities (§3.8).
- When building the notifications mark-as-read mechanism, match the frontend's committed contract: POST /notifications/{id}/read per notification id (§14).
- When building GET /organizations/me/dashboard , filter recentActivity to pending-status participations only, not all statuses (§15).
- **NEW (2026-08-14)** — optionally support ?unread=true on GET /notifications ; not required since the frontend filters defensively, but recommended given the 5-second polling interval (§14 update).
- **NEW (2026-08-14)** — add email to the nested volunteer object on GET /opportunities/{id}/participants (sourced from users.email), alongside the already-expected volunteer.phone and volunteer.photo — required, the frontend now renders it directly with no fallback (§17.4 update).

End of addendum. All findings above trace to specific frontend source files as of today; none were inferred beyond what the code and its inline comments state explicitly. Anything from the reference document not mentioned here is unchanged. Entries marked 2026-08-14 were appended in a follow-up revision after the initial pass; all other entries are unchanged from that initial pass.

Backend Requirements Addendum — Update (Photo/Logo Upload)

Volunteer Organization Platform — continuation of Backend-Requirements-Addendum.pdf

Scope: this page covers only the profile-photo / organization-logo upload flow, confirmed today in a direct conversation with the backend team. Reference: this addendum's own §17.4 (volunteer email) and the original Backend-Requirements-Document.pdf. Section numbers below continue the same §-numbering convention as the rest of this addendum.

§17.6 Profile photo / logo upload — confirmed backend scope

Backend team confirmed verbally that photo/logo upload will be fully supported across the project (this was previously declined when first proposed). Two separate, independent gaps were found by direct inspection of the current backend code. **Both are required** — they are unrelated bugs in different controllers, and fixing one does not fix or depend on the other. Frontend is already built and ready for both; no further frontend work is required once each ships.

Gap	Status	Current backend behavior (confirmed by code)	Required fix
A. Volunteer photo field shape VolunteerResource.php	Confirmed by backend response will be a URL, per Spatie convention	<code>'photo' => \$this->getFirstMedia('profile_photo') ? : null</code> <code>getFirstMedia()</code> returns a full Spatie Media model object, not a URL string.	Change to <code>getFirstMediaUrl('profile_photo')</code> — same pattern already used correctly in <code>OrganizationResource.php</code> for <code>profile_image</code> . One-line fix.
B. Organization logo on update OrganizationController.php	Still required not yet explicitly confirmed by backend	<code>store()</code> (registration) already handles <code>hasFile('photo')</code> correctly. <code>update()</code> only runs <code>\$organization->update(\$request->validated())</code> — no file handling at all. An organization cannot change its logo after registration today.	Add the same <code>hasFile('photo')</code> handling already used in <code>store()</code> to <code>update()</code> : clear the existing <code>profile_image</code> media collection and add the new file, matching <code>VolunteerController::update()</code> 's existing pattern for <code>profile_photo</code> .

Backend action — both required, independently of each other:

- VolunteerResource.php — swap `getFirstMedia` → `getFirstMediaUrl` for the `photo` field (Gap A) — **confirmed in progress**.
- OrganizationController::update() — add `hasFile('photo')` handling to the `profile_image` media collection, mirroring `VolunteerController::update()` (Gap B) — **still needs explicit confirmation**. Without this, organizations remain unable to change their logo after registration even after Gap A ships. OrganizationRequest.php will also need a `photo` validation rule added (currently has none — see original Backend-Requirements-Document.pdf's request-rules table).

Frontend readiness — confirmed, no action needed once shipped

Endpoint	Frontend status	Notes
<code>POST /volunteers/{id}</code> (<code>_method=PUT</code> , multipart)	Ready	Already sends <code>photo</code> as a file. Response parsing already defensively handles both a plain URL string and a raw Media object (for Gap A above), so it will pick up the fix automatically — no frontend change needed the moment Gap A ships.
<code>POST /organizations/{id}</code> (<code>_method=PUT</code> , multipart)	Sending, backend not yet processing	Frontend already sends <code>photo</code> as a file on every save. Backend currently ignores it silently (no error, no crash) until Gap B ships — logo changes just won't persist until then. Reads the response's <code>profile_image</code> field already, ready the moment it's populated.

Mock mode (local frontend testing without a live backend) already fully simulates both flows today, storing photos as base64 Data URLs so they survive page reloads — this was a separate, already-fixed frontend-only issue unrelated to the two backend gaps above.